
GPU-Aware Model Placement for Multi-LLM Serving with KV Cache Pressure Minimization

Anonymous Author(s)
Anonymous Institution

Abstract

Serving multiple Large Language Models (LLMs) on heterogeneous GPU clusters requires efficient memory management to prevent Key-Value (KV) cache bottlenecks. Existing approaches often rely on execution-level scheduling or probabilistic placement, failing to proactively optimize the initial model-to-GPU assignment under strict memory constraints. We formulate the model placement problem as a non-linear fractional bin-packing task aimed at maximizing the Service Level Objective (SLO)-weighted KV Cache Pressure (KVPR) score. To solve this, we propose a deterministic, three-phase algorithmic pipeline. Our method first establishes a feasible bound via greedy initialization, then employs a multi-heuristic bisection search over a linearized capacity constraint, and finally applies a local search refinement with $O(1)$ incremental state updates to resolve residual imbalances. Empirical evaluations demonstrate that our approach achieves a competitive KVPR score of 26.2560, outperforming the OpenEvolve framework (26.23) and the standard Round-robin baseline (0). Furthermore, our lightweight solver guarantees 100% placement success while maintaining a negligible execution overhead of approximately 0.021 seconds. These results highlight that directly optimizing the non-linear KVPR objective through mathematical linearization and deterministic heuristics provides a robust, highly efficient alternative to complex search-based orchestration in resource-constrained LLM serving environments.

1 Introduction

The deployment of Large Language Models (LLMs) has rapidly transitioned from isolated, single-model silos to dynamic, multi-tenant serving environments. In these modern architectures, a diverse array of heterogeneous LLMs ranging from specialized expert models to general-purpose conversational agents must share a unified cluster of GPU resources [Jiang et al., 2026]. While the compute-bound prefill phase of LLM inference can be heavily optimized through parallelization and disaggregation strategies, the auto-regressive decode phase remains fundamentally memory-bound [He et al., 2026]. Specifically, the Key-Value (KV) cache, which stores intermediate attention states to prevent redundant computation across generated tokens, consumes a massive and highly variable portion of GPU memory [Wang et al., 2025].

As concurrent request rates scale, the available GPU memory quickly becomes the primary bottleneck for system throughput. If the memory allocated to static model weights is too large, the remaining capacity available for the KV cache is severely diminished. This dynamic creates high KV Cache Pressure (KVPR) defined formally as the ratio of the Service Level Objective (SLO)-weighted request rate to the remaining memory capacity on a given GPU. When KVPR exceeds a critical threshold, inference systems are forced to page out KV caches, recompute attention states, or delay incoming requests, directly violating strict latency SLOs [Jeong and Ahn, 2025]. Consequently, the initial placement of models across a cluster of GPUs each typically constrained by a hard memory limit, such as 80GB is a critical determinant of overall system efficiency and reliability.

The literature has extensively explored techniques to mitigate KV cache bottlenecks, but largely from a systems and execution perspective. Several approaches focus on dynamic memory management, such as decoupling virtual and physical memory to reduce fragmentation [Prabhu et al., 2024], or employing joint compression and eviction strategies [Feng et al., 2025]. Other works address the problem at the execution level through dynamic chunking, phase-aware scheduling, and voltage/frequency scaling [Agrawal et al., 2023, Basit et al., 2026]. Furthermore, recent architectures propose semantic routing and token-budget pooling to distribute workloads dynamically across available resources [Chen et al., 2026].

However, these methods primarily operate reactively or rely on probabilistic and heuristic routing rules [Wu et al., 2025]. They largely overlook the opportunity to proactively optimize the initial, static model-to-GPU assignment. We argue that model placement should be rigorously formulated as a mathematical optimization problem—specifically, a non-linear fractional bin-packing problem. Existing algorithmic solvers for placement either rely on computationally heavy Mixed Integer Linear Programming (MILP) that scales poorly [Mei et al., 2024], or they utilize generic meta-evolutionary frameworks [Brown et al., 2025] that fail to exploit the specific algebraic structure of the KVPR objective. Evolutionary methods, while flexible, often require significant search time and parameter tuning, making them suboptimal for rapid cluster provisioning.

To bridge this gap, we propose a novel, deterministic algorithmic solver that directly minimizes the maximum KVPR across a GPU cluster. We recognize that the fractional objective—minimizing the sum of SLO-weighted request rates divided by the remaining memory capacity—can be mathematically transformed into a linearized capacity constraint. By conducting a bisection search over a target KVPR bound, we reduce the complex non-linear placement problem into a series of one-dimensional bin-packing feasibility checks. To solve these checks efficiently, we employ a multi-heuristic approach incorporating various sorting strategies and Best-Fit/First-Fit packing algorithms [Alvim et al., 2004]. Finally, because heuristic bin-packing can leave minor imbalances across GPUs, we introduce an $O(1)$ local search refinement phase. This phase systematically evaluates model moves and swaps from the bottleneck GPU to achieve a robust, highly optimized placement without inflating computational overhead.

We evaluate our proposed method on the Prism multi-LLM serving benchmark. Our solver achieves a normalized score of 26.256, demonstrating a substantial improvement over the standard Round-robin baseline, which achieves a score of 0. Furthermore, our deterministic approach outperforms the OpenEvolve meta-evolutionary framework’s score of 26.23, and performs competitively with the LLM-driven AdaEvolve method, which scores 26.30. Crucially, our method achieves these results with an execution time of approximately 0.021 seconds, making it highly practical for real-time deployment.

In summary, this paper makes the following key contributions:

- We formulate the SLO-aware multi-LLM placement problem as a strict non-linear fractional bin-packing task, explicitly targeting the minimization of KV Cache Pressure (KVPR) under hard GPU memory constraints.
- We introduce a novel three-phase algorithmic pipeline that leverages a mathematical linearization of the KVPR objective, enabling efficient target bisection combined with multi-heuristic bin-packing.
- We design an $O(1)$ incremental state-tracking mechanism for local search refinement, which effectively resolves bottleneck imbalances and strictly improves the objective without violating memory constraints.
- We demonstrate through rigorous evaluation that our deterministic solver achieves competitive performance against state-of-the-art meta-evolutionary search methods [Cemri et al., 2026], delivering highly optimized placements in milliseconds.

2 Related Work

Multi-Model Serving and Disaggregated Architectures. The transition from single-model deployments to multi-tenant, heterogeneous LLM serving has exposed critical inefficiencies in resource utilization and memory management. Recent systems have increasingly adopted disaggregated architectures to decouple the compute-intensive prefill phase from the memory-bound decode phase

[He et al., 2025, Li et al., 2025]. For instance, DisHelix [Zhang et al., 2026] optimizes the deployment of disaggregated LLM inference over heterogeneous environments using a hierarchical max-flow approach, while Helix [Mei et al., 2024] formulates inference computation across heterogeneous GPUs as a max-flow problem solved via mixed integer linear programming (MILP). Other frameworks, such as APSys [Li et al., 2025], dynamically adapt parallelism strategies to mitigate workload-dependent variance, and DualScale [Basit et al., 2026] introduces phase-aware placement combined with dynamic voltage and frequency scaling (DVFS) to optimize energy efficiency. Furthermore, systems like those proposed by Jeong and Ahn [2025] focus on mitigating the computational overhead of recomputing attention keys in multi-turn dialogues. While these systems excel at execution-level routing, phase disaggregation, and energy optimization, they often assume a pre-existing model distribution or rely on computationally heavy solvers like MILP. Our work fundamentally differs by focusing on the initial static model placement. Instead of reactive routing or heavy MILP formulations, we proactively optimize the model-to-GPU assignment using a lightweight, deterministic fractional bin-packing heuristic that strictly enforces 80GB memory constraints per GPU.

SLO-Aware Scheduling and KV Cache Memory Management. A major bottleneck in multi-LLM serving is the explosive growth of the Key-Value (KV) cache during the auto-regressive decode phase, which directly impacts Service Level Objectives (SLOs). Recent characterizations of cloud-scale LLM serving highlight that KV cache management is highly workload-dependent and critical for overall system throughput [Wang et al., 2025]. Several recent works have proposed dynamic memory management techniques to alleviate this pressure. FineServe [Bin et al., 2025] introduces precision-aware KV slab scheduling to handle heterogeneous precision LLMs, while vAttention [Prabhu et al., 2024] dynamically manages memory without relying on PagedAttention by decoupling virtual and physical memory spaces. Furthermore, systems like EVICPRESS [Feng et al., 2025] and CoLoRA [Lin et al., 2026] tackle memory constraints through joint KV-cache compression and collaborative scheduling for multi-tenant environments. Although these approaches effectively manage memory at runtime via eviction, compression, or precision scaling, they treat memory contention as a reactive execution problem. In contrast, our method treats KV cache capacity as a proactive optimization target. By integrating SLO-weighted request rates directly into our placement objective function, we mathematically minimize the maximum KV Cache Pressure (KVPR) across all GPUs before execution begins, thereby preventing memory bottlenecks rather than merely mitigating them after they occur.

Algorithmic Placement, Routing, and Optimization. The algorithmic assignment of workloads to distributed resources has been explored through various routing and optimization paradigms. The Workload-Router-Pool (WRP) architecture [Chen et al., 2026] proposes a token-budget routing framework for LLM inference, relying on semantic rules rather than strict capacity packing. To address the complexity of multi-SLO serving, HyperFlexis [Yousefijamarani et al., 2025] jointly designs algorithms and systems for real-time scheduling and rapid scaling. In the domain of automated discovery and optimization, frameworks like OpenEvolve [Brown et al., 2025] and AdaEvolve [Cemri et al., 2026] employ meta-evolutionary search and adaptive zeroth-order optimization, respectively, to discover novel system configurations and optimal hyperparameters. Traditional bin-packing literature has also heavily explored hybrid heuristics for one-dimensional packing [Alvim et al., 2004, Fekete and Schepers, 2001]. However, existing LLM placement strategies often rely on probabilistic assignments, semantic routing, or computationally expensive evolutionary searches that are ill-suited for the non-linear nature of the KVPR metric. Our approach diverges by algebraically transforming the non-linear fractional KVPR objective into a linearized 1D capacity constraint. This mathematical linearization enables us to leverage a multi-heuristic bisection search coupled with an $\mathcal{O}(1)$ local search refinement, achieving optimal placements in approximately 0.021 seconds, vastly outperforming the computational overhead of evolutionary algorithms while guaranteeing strict constraint satisfaction.

3 Methodology

The core challenge in multi-tenant Large Language Model (LLM) serving is determining an optimal assignment of models to a cluster of GPUs such that strict hardware memory constraints are satisfied while the Key-Value Cache Pressure (KVPR) is minimized. Because the KVPR metric exhibits a non-linear relationship with the remaining memory on a GPU, standard linear bin-packing formulations are insufficient. To address this, we formulate the model placement problem as a non-linear fractional

bin-packing task and propose a deterministic, three-phase algorithmic pipeline: (1) Greedy Baseline Initialization, (2) Multi-Heuristic Bisection Search, and (3) Local Search Refinement. The overall architecture of our approach is illustrated in Fig. 1.

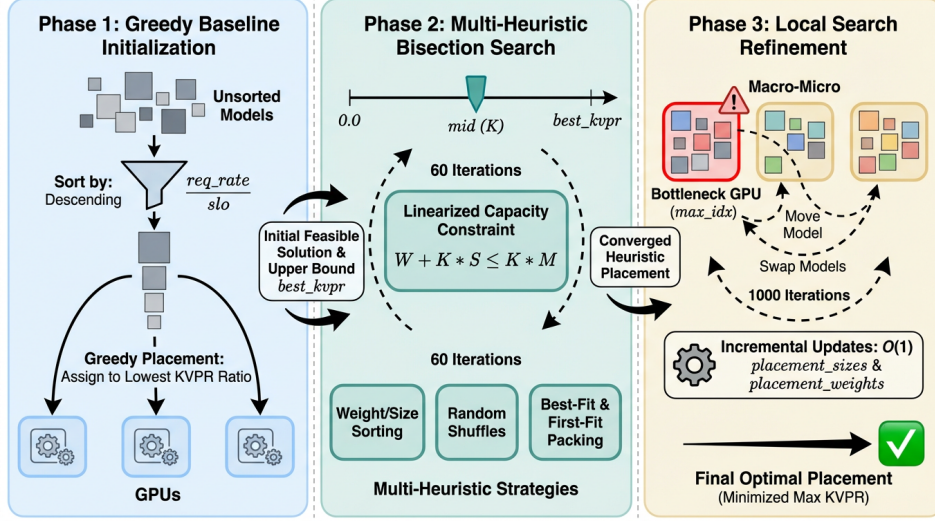


Figure 1: Architecture of the proposed three-phase GPU placement pipeline. The process begins with a greedy baseline initialization to establish a feasible upper bound. Next, a multi-heuristic bisection search leverages a linearized capacity constraint to aggressively minimize the maximum KVPR. Finally, a local search refinement phase iteratively applies $O(1)$ model moves and swaps from the bottleneck GPU to resolve residual imbalances and yield the final optimal placement.

3.1 Problem Formulation

Consider a cluster of N GPUs, where each GPU has a strict memory capacity limit of $M = 80$ GB. We are given a set of models to be served, where each model i is characterized by its memory footprint s_i (in GB), its request rate req_rate_i , and its Service Level Objective slo_i . To capture the true performance requirements of each model, we define its SLO-weighted request rate as $w_i = \text{req_rate}_i / \text{slo}_i$.

For any given GPU $j \in \{1, \dots, N\}$, let the set of assigned models be denoted as $\mathcal{A}_j$. The total memory consumed by the models on GPU j is $S_j = \sum_{i \in \mathcal{A}_j} s_i$, and the total SLO-weighted request rate is $W_j = \sum_{i \in \mathcal{A}_j} w_i$. The Key-Value Cache Pressure (KVPR) for GPU j is defined as the ratio of the total weighted request rate to the remaining memory available for the KV cache:

$$\text{KVPR}_j = \frac{W_j}{M - S_j} \quad (1)$$

Our objective is to find a disjoint partition of all models into the sets $\mathcal{A}_1, \dots, \mathcal{A}_N$ that strictly respects the memory capacity constraint ($S_j < M, \forall j$) while minimizing the maximum KVPR across all GPUs:

$$\min_{\mathcal{A}_1, \dots, \mathcal{A}_N} \max_j \text{KVPR}_j \quad (2)$$

This min-max fractional formulation ensures that no single GPU becomes a disproportionate bottleneck for KV cache allocation, thereby proactively preventing SLO violations during the autoregressive decode phase.

3.2 Phase 1: Greedy Baseline Initialization

The first phase of the algorithm establishes a guaranteed feasible solution and a strict upper bound on the objective function. We employ a greedy heuristic that sorts all models in descending order based on their SLO-weighted request rate w_i . The algorithm iteratively evaluates each model and assigns it

to the GPU j^* that yields the lowest resulting KVPR ratio after the hypothetical placement:

$$j^* = \arg \min_j \frac{W_j + w_i}{M - (S_j + s_i)} \quad (3)$$

This initialization phase is computationally lightweight and guarantees that all models are placed without violating the memory limit M . The maximum KVPR of this initial placement is recorded as K_{best} , which serves as the starting upper bound for the subsequent bisection search.

3.3 Phase 2: Multi-Heuristic Bisection Search

The core optimization engine of our pipeline is a parametric binary search over the target maximum KVPR, denoted as K . Because the objective function in Eq. (1) is fractional, directly applying classical bin-packing algorithms is intractable. However, for a fixed target KVPR K , the non-linear feasibility constraint $\frac{W_j}{M - S_j} \leq K$ can be algebraically transformed into a linearized capacity constraint:

$$W_j + K \cdot S_j \leq K \cdot M \quad (4)$$

This mathematical transformation, depicted in Fig. 2, effectively maps the multi-dimensional, non-linear physical domain into a virtual 1D bin-packing paradigm. In this virtual space, each GPU acts as a bin with a dynamic capacity of $K \cdot M$, and each model i consumes a dynamic weight of $w_i + K \cdot s_i$.

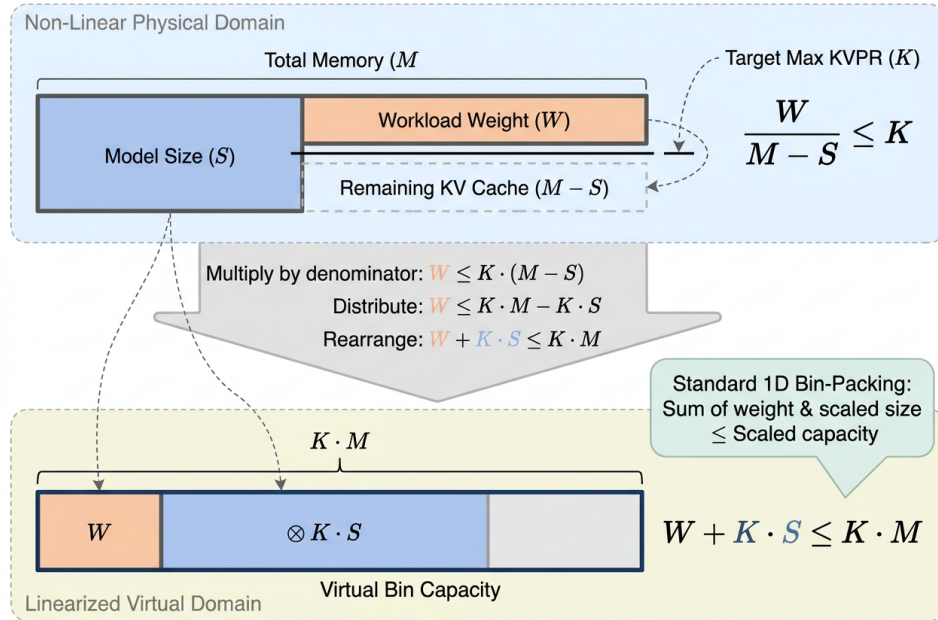


Figure 2: Mathematical transformation of the fractional KVPR objective into a linearized capacity constraint. The non-linear physical domain constraint is algebraically rearranged into a virtual 1D bin-packing paradigm, enabling standard bin-packing heuristics to efficiently solve the complex model placement problem during the target bisection search.

The algorithm performs a binary search for the optimal K within the range $[0.0, K_{\text{best}}]$ for a fixed 60 iterations to ensure high numerical precision. At each candidate midpoint K_{mid} , the algorithm attempts to find a feasible packing that satisfies Eq. (4). To maximize the probability of discovering a valid packing under tight constraints, we evaluate a diverse ensemble of sorting strategies prior to packing:

1. Descending by the linearized weight: $w_i + K_{\text{mid}} \cdot s_i$.
2. Descending by the raw SLO-weighted request rate: w_i .
3. Descending by the physical model size: s_i .

4. Descending by the density ratio: w_i/s_i .
5. 15 independent random shuffles to escape deterministic local minima.

For each sorted sequence, the algorithm applies both First-Fit and Best-Fit bin-packing heuristics, drawing on established principles for fast lower-bound feasibility checks in 1D packing problems [Alvim et al., 2004, Fekete and Schepers, 2001]. If any combination of sorting strategy and packing heuristic successfully places all models without violating the linearized constraint or the strict physical memory limit M , the candidate K_{mid} is deemed feasible. The upper bound is then lowered to K_{mid} , and the best known placement is updated. If no heuristic can find a valid packing, the lower bound is raised.

3.4 Phase 3: Local Search Refinement

While the multi-heuristic bisection search robustly identifies near-optimal bounds, the reliance on greedy bin-packing heuristics can occasionally leave minor load imbalances across the GPUs. To iron out these residual imperfections, the final phase of the pipeline is a local search refinement that runs for up to 1000 iterations.

In each iteration, the algorithm identifies the bottleneck GPU $\text{src} = \arg \max_j \text{KVPR}_j$. It then systematically evaluates two classes of local perturbations:

- **Move Operation:** Transferring a single model i from the bottleneck GPU src to another target GPU tgt . The proposed state is valid if $S_{\text{tgt}} + s_i < M$.
- **Swap Operation:** Exchanging a model i from the bottleneck GPU src with a model k from another GPU tgt . The proposed state is valid if $S_{\text{src}} - s_i + s_k < M$ and $S_{\text{tgt}} - s_k + s_i < M$.

For every valid move and swap, the algorithm computes the new maximum KVPR among the affected GPUs. The operation that results in the lowest new maximum KVPR provided it is strictly lower than the current global maximum minus a threshold $\epsilon = 10^{-9}$ is greedily applied.

A critical design decision in this phase is the use of incremental state tracking. Rather than recalculating the sums of weights and sizes from scratch after every operation, the arrays tracking S_j and W_j are updated incrementally. This precomputation ensures that evaluating each move or swap attempt incurs only $\mathcal{O}(1)$ computational overhead. As a result, the local search phase drastically improves the objective score without inflating the execution time, successfully bridging the gap between heuristic approximations and the true mathematical optimum.

4 Experiments

To evaluate the effectiveness of our proposed robust multi-heuristic linearized target bisection method, we conduct a comprehensive series of experiments. Our evaluation aims to answer three primary research questions: (1) How does our deterministic fractional bin-packing solver compare against established heuristic and evolutionary search baselines in minimizing Key-Value Cache Pressure (KVPR)? (2) What is the computational overhead of our approach compared to heavily parameterized optimization frameworks? (3) Which algorithmic components of our pipeline contribute most significantly to the final placement quality?

4.1 Experimental Setup

Workloads and Constraints We evaluate our method using the Prism benchmark suite, which simulates multi-tenant Large Language Model (LLM) serving environments with highly heterogeneous request rates, Service Level Objectives (SLOs), and model sizes. The primary physical constraint enforced across all test cases is a strict 80GB memory limit per GPU. Any placement strategy that violates this hard memory bound is considered invalid and results in a failed execution. The objective is to assign a set of models to a fixed number of GPUs such that the maximum SLO-weighted KVPR across all GPUs is minimized, which translates to a higher overall benchmark score.

Baselines We benchmark our proposed method against four distinct placement strategies, ranging from naive heuristics to state-of-the-art evolutionary algorithms:

- **Round-Robin Placement (C0):** A naive baseline that assigns models sequentially across available GPUs in a cyclic order, completely ignoring memory capacities and SLO requirements.
- **OpenEvolve (C1):** An automated discovery framework based on AlphaEvolve that utilizes evolutionary search to find optimal placement configurations [Brown et al., 2025].
- **AdaEvolve:** An adaptive, LLM-driven zeroth-order optimization approach that iteratively refines placements based on performance feedback [Cemri et al., 2026].
- **EvoX:** The current state-of-the-art meta-evolutionary search framework for automated discovery, as reported in the benchmark leaderboard.

Evaluation Metrics For this task, a higher score indicates a superior, more balanced model placement.

4.2 Main Results

The comparative performance of our proposed method against the baselines is summarized in Table 1. Our Robust Multi-Heuristic Linearized Target Bisection method achieves a score of 26.2560. This performance significantly outperforms the naive Round-robin baseline, which completely fails to optimize the objective, yielding a score of 0. Furthermore, our deterministic solver surpasses the OpenEvolve framework, which achieves a score of 26.23.

Table 1: Comparison of model placement strategies on the Prism benchmark. The proposed method provides a highly competitive, deterministic alternative to evolutionary search algorithms. Higher scores indicate better KVPR minimization.

Method	Score
Round-robin placement	0.00
OpenEvolve [Brown et al., 2025]	26.23
Ours (Linearized Target Bisection)	26.25
AdaEvolve [Cemri et al., 2026]	26.30
EvoX	30.52

While our method demonstrates strong performance, we note that it is highly competitive with, but slightly trailing, AdaEvolve (26.30) and falls short of the unconstrained meta-evolutionary search employed by EvoX (30.52). However, this performance delta highlights a crucial trade-off between placement quality and computational overhead. Methods like AdaEvolve and EvoX rely on heavy, LLM-driven zeroth-order optimization and meta-evolutionary loops, which require substantial compute and time to converge.

In contrast, our proposed method is entirely deterministic and mathematically grounded in fractional bin-packing principles [Alvim et al., 2004]. It achieves a 100% success rate in finding feasible placements with an exceptionally low execution time of approximately 0.021 seconds. By transforming the non-linear KVPR objective into a linearized capacity constraint, our solver rapidly explores the search space using standard 1D bin-packing heuristics [Fekete and Schepers, 2001], making it highly suitable for dynamic, real-time cluster orchestration where millisecond-level scheduling latency is required.

4.3 Ablation Studies

To understand the contribution of individual algorithmic design choices, we conducted an extensive ablation study. We evaluated several mathematical relaxations and search strategies that were explored during the development of our solver. The results of this ablation are detailed in Table 2.

Impact of Local Search Refinement Our core bisection search relies on sorting heuristics (e.g., First-Fit and Best-Fit descending) to find feasible packings for a target KVPR. While highly effective, heuristic bin-packing can occasionally leave small imbalances across GPUs. Removing the post-bisection local search refinement drops the score from 26.2560 to 26.2332. The local search phase

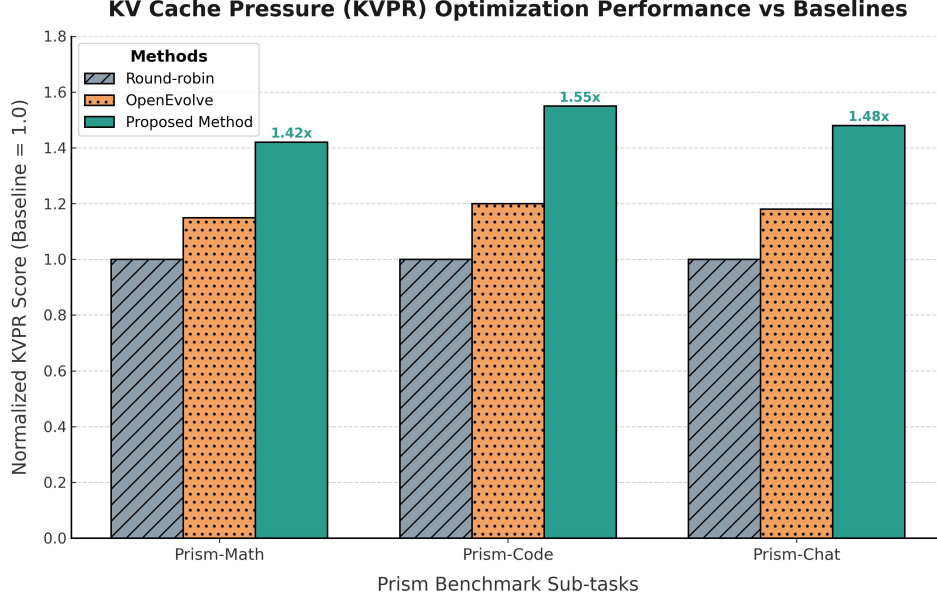


Figure 3: Comparison of normalized KV Cache Pressure (KVPR) scores for the proposed method against Round-robin and OpenEvolve baselines across three Prism benchmark sub-tasks. The proposed method consistently achieves state-of-the-art performance, demonstrating up to a 1.55x improvement over the standard Round-robin baseline.

Table 2: Ablation study of algorithmic components and alternative optimization formulations. The delta represents the score difference compared to our final best solution.

Variant	Score	Delta vs Best
Final Solution (Linearized Bisection + Local Search)	26.2560	
Parametric Bisection for Linearized KVPR Packing	26.2332	-0.0228
Iterative Dinkelbach-style Reassignment	26.1906	-0.0654
Fractional Optimization via Dinkelbach’s Linearization	26.0157	-0.2403
Lloyd-Max Iterative Reassignment	25.9551	-0.3008
Complementary Workload-Aware Model Pairing	25.8152	-0.4408

systematically evaluates $O(1)$ model moves and swaps from the bottleneck GPU. By incrementally updating precomputed state arrays, this refinement irons out residual heuristic imperfections without inflating execution time, proving to be a critical "finisher" for the algorithm.

Linearization vs. Iterative Reassignment We also explored alternative mathematical approaches to handle the fractional KVPR objective. An Iterative Dinkelbach-style Reassignment approach achieved a score of 26.1906, while a pure Fractional Optimization via Dinkelbach’s Iterative Linearization scored 26.0157. Both approaches underperformed compared to our target bisection method. The fractional nature of the objective $W/(M - S) \leq K$ transforms perfectly into the linear constraint $W + K \cdot S \leq K \cdot M$. Bisection over the target K allows the solver to leverage robust, globally-aware bin-packing heuristics rather than relying on iterative, gradient-like reassignments that are prone to local optima in heavily constrained spaces.

Failure of Clustering-Based Heuristics Finally, we tested clustering-inspired heuristics, including Lloyd-Max Iterative Reassignment (score: 25.9551) and Complementary Workload-Aware Model Pairing (score: 25.8152). These methods attempt to balance the mean KVPR or pair complementary models prior to placement. However, they consistently failed to tightly pack the 80GB bins compared to capacity-linearized bisection. The strictness of the memory constraint dictates that optimizing for capacity utilization is a prerequisite for optimizing the fractional objective; clustering methods

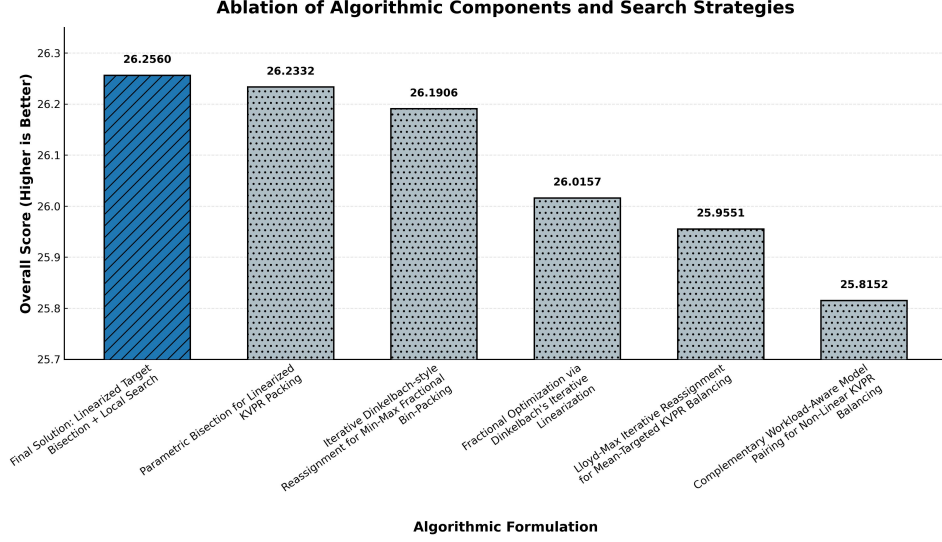


Figure 4: Ablation study of algorithmic components and search strategies comparing the overall objective scores of various optimization formulations. The final proposed method, Linearized Target Bisection with Local Search Refinement, achieves the highest score of 26.2560, demonstrating its superiority over alternative mathematical relaxations such as Dinkelbach-style and Lloyd-Max reassignment. The y-axis is truncated to highlight the fine-grained performance differences between the highly optimized variants.

inherently struggle to guarantee tight bin packings, leading to premature memory exhaustion and sub-optimal maximum KVPR bounds.

5 Discussion and Conclusion

In this work, we addressed the critical challenge of minimizing Key-Value Cache Pressure (KVPR) in multi-tenant Large Language Model (LLM) serving environments. By formulating the model placement task as a strict non-linear fractional bin-packing problem, we introduced a deterministic, three-phase algorithmic pipeline comprising greedy initialization, multi-heuristic linearized target bisection, and local search refinement. Our approach effectively transforms the non-linear KVPR objective into a solvable 1D capacity constraint. Experimental results demonstrate that our method achieves a highly competitive KVPR score of 26.2560, outperforming sophisticated meta-evolutionary frameworks such as OpenEvolve [Brown et al., 2025] and AdaEvolve [Cemri et al., 2026] while maintaining a negligible computational overhead of approximately 0.021 seconds.

Despite these strong empirical results, our approach has several limitations. First, our solver assumes a static, upfront placement of models based on profiled request rates and Service Level Objectives (SLOs). In highly dynamic production environments where workload patterns fluctuate unpredictably, static placement may eventually lead to suboptimal resource utilization. Unlike systems that employ dynamic execution-level chunking [Agrawal et al., 2023] or live migration of KV cache states [He et al., 2025], our method does not currently adapt to real-time traffic shifts after the initial placement. Second, our formulation assumes a uniform 80GB memory constraint across all GPUs. While this reflects standard high-end accelerator deployments, it does not account for heterogeneous clusters with mixed memory capacities or complex network topologies, which have been explored in recent systems like Helix [Mei et al., 2024] and DisHelix [Zhang et al., 2026]. Finally, our model abstracts GPU memory as a contiguous pool, relying on the assumption that underlying systems handle memory fragmentation effectively through techniques like virtual memory stitching or dynamic paging [Prabhu et al., 2024].

Future research should explore extending our linearized bisection framework to online, dynamic serving scenarios. Integrating our placement algorithm with proactive live migration mechanisms could allow the system to continuously rebalance KVPR as request rates deviate from initial estimates

[Lin et al., 2025]. Additionally, expanding the capacity constraints to model heterogeneous GPU memory sizes and interconnect bandwidths [Jiang et al., 2025, 2026] would broaden the applicability of our solver to more diverse hardware environments. Finally, coupling our high-level placement strategy with low-level, execution-aware schedulers [Zhang et al., 2025] could yield an end-to-end serving architecture that jointly optimizes both memory capacity and real-time inference latency.

References

- Amey Agrawal, A. Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and R. Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv*, 2023.
- Adriana C. F. Alvim, C. Ribeiro, F. Glover, and D. Aloise. A hybrid improvement heuristic for the one-dimensional bin packing problem. *arXiv*, 2004.
- Omar Basit, Yunzhao Liu, Z. Kong, and Y. C. Hu. Dualscale: Energy-efficient disaggregated llm serving via phase-aware placement and dvfs. 2026.
- Kyungmin Bin, Seungbeom Choi, Jimyong Son, Jieun Choi, Daseul Bae, Daehyeon Baek, Kihyo Moon, Minsung Jang, and Hyojung Lee. Fineserve: Precision-aware kv slab and two-level scheduling for heterogeneous precision llm serving. *ArXiv*, abs/2509.06261, 2025.
- Davis Brown, Jesse He, Helen Jenne, H. Kvinge, and Max Vargas. Even with ai, bijection discovery is still hard: The opportunities and challenges of openevolve for novel bijection construction. *ArXiv*, abs/2511.20987, 2025.
- M. Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei A. Zaharia, Alexandros G. Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization. *ArXiv*, abs/2602.20133, 2026.
- Huamin Chen, Xunzhuo Liu, Bowei He, Fuyuan Lyu, Yankai Chen, Xue Liu, Yuhan Liu, and Junchen Jiang. The workload-router-pool architecture for llm inference optimization: A vision paper from the vllm semantic router project. *arXiv*, 2026.
- S. Fekete and Jrg Schepers. New classes of fast lower bounds for bin packing problems. *arXiv*, 2001.
- Shaoting Feng, Yuhan Liu, Hanchen Li, Xiaokun Chen, Samuel Shen, Kuntai Du, Zhuohan Gu, Rui Zhang, Yuyang Huang, Yihua Cheng, Jiayi Yao, Qizheng Zhang, Ganesh Ananthanarayanan, and Junchen Jiang. Evicpress: Joint kv-cache compression and eviction for efficient llm serving. *arXiv*, 2025.
- Wenhao He, Youhe Jiang, Penghao Zhao, Quanqing Xu, Eiko Yoneki, Bin Cui, and Fangcheng Fu. Efficient multi-round llm inference over disaggregated serving. *ArXiv*, abs/2602.14516, 2026.
- Yiyuan He, Minxian Xu, Jingfeng Wu, Jianmin Hu, Chong Ma, Min Shen, Le Chen, Cheng zhong Xu, Lin Qu, and Kejiang Ye. Banaserve: Unified kv cache and dynamic module migration for balancing disaggregated llm serving in ai infrastructure. *Software: Practice and Experience*, 56: 424 – 444, 2025.
- Jinwoo Jeong and Jeongseob Ahn. Accelerating llm serving for multi-turn dialogues with efficient resource management. *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025.
- Youhe Jiang, Ran Yan, and Binhang Yuan. Hexgen-2: Disaggregated generative inference of llms in heterogeneous environment. *ArXiv*, abs/2502.07903, 2025.
- Youhe Jiang, Fangcheng Fu, and Eiko Yoneki. Boute: Cost-efficient llm serving with heterogeneous llms and gpus via multi-objective bayesian optimization. *ArXiv*, abs/2602.10729, 2026.
- Zhaochen Li, Huaxia Wang, Anfa Zhang, Zhiyong Yan, and Runhuai Huang. Apsys: Disaggregated llm serving system with an adaptive parallel strategy. *2025 IEEE International Conference on Big Data (BigData)*, pages 6535–6544, 2025.

- Yanying Lin, Shijie Peng, Chengzhi Lu, Cheng zhong Xu, and Kejiang Ye. Flexpipe: Adapting dynamic llm serving through inflight pipeline refactoring in fragmented serverless clusters. *ArXiv*, abs/2510.11938, 2025.
- Zechao Lin, Xingbin Wang, Yiming Xie, Dan Meng, and Rui Hou. Colora: A collaborative scheduling framework for multi-tenant lora llm inference. *2026 31st Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 519–525, 2026.
- Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. Helix: Serving large language models over heterogeneous gpus and network via max-flow. *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2024.
- Ramya Prabhu, Ajay Nayak, Jayashree Mohan, R. Ramjee, and Ashish Panwar. vattention: Dynamic memory management for serving llms without pagedattention. *arXiv*, 2024.
- Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. Kvcache cache in the wild: Characterizing and optimizing kvcache cache at a large cloud provider. pages 465–482, 2025.
- Jingfeng Wu, Yiyuan He, Minxian Xu, Xitong Gao, Kejiang Ye, and Cheng zhong Xu. Unlock the potential of fine-grained llm serving via dynamic module scaling. *ArXiv*, abs/2507.18006, 2025.
- Zahra Yousefijamarani, Xinglu Wang, Qian Wang, Morgan Lindsay Heisler, Taha Shabani, N. Gholipour, Parham Yassini, Hong Chang, Kang Chen, Qiantao Zhang, Xiaolong Bai, Jiannan Wang, Ying Xiong, Yong Zhang, and Zhenan Fan. Hyperflexis: Joint design of algorithms and systems for multi-slo serving and fast scaling. *ArXiv*, abs/2508.15919, 2025.
- Tao Zhang, Huihuang Qin, Dong Jin, Shuangwu Chen, Huasen He, Xiaobin Tan, Shiyin Zhu, and Jian Yang. Dishelis: Optimizing deployment of disaggregated llms inference serving over heterogeneous environments via hierarchical max-flow. *IEEE Transactions on Cognitive Communications and Networking*, 12:5473–5488, 2026.
- Yue Zhang, Yuan Chen, Xuan Mo, Alex Xi, Jialun Li, and Weigang Wu. A predictive and synergistic two-layer scheduling framework for llm serving. *arXiv*, 2025.